مستندات فاز اول پروژه
سامانه بازیابی محصول ترب با RAG

پروژه بازیابی محصول ترب

۱۵ خرداد ۱۴۰۵

چکیده

هدف فاز اول، ساخت یک پایپلاین قابل بازتولید برای بازیابی محصول روی داده‌های ترب است. این فاز شامل شناخت دیتاست، تحلیل اکتشافی داده، پاک‌سازی و یکپارچه‌سازی، ساخت متن قابل embedding، تولید هایembedding متنی و تصویری، ذخیره آن‌ها در FAISS، پیاده‌سازی سه خط پایه بازیابی متنی، تصویری و چندوجهی، و ارزیابی خروجی‌ها است. این گزارش توضیح می‌دهد که هر بخش از مستند فاز اول چگونه در کد و artifacts پروژه پیاده‌سازی شده است.

فهرست مطالب

۱ محل قرارگیری پروژه ۲

۲ خلاصه نیازمندی‌ها و پیاده‌سازی ۲

۳ معماری کلی فاز اول ۳

۴ ساختار داده‌ها ۳

۵ تحلیل اکتشافی داده ۴

۶ پاک‌سازی و یکپارچه‌سازی ۴

۷ استراتژی subset ۵

۸ بازیابی متنی ۵

۹ ارزیابی بازیابی متنی ۵

۱۰ بازیابی تصویری ۶

۱۱ بازیابی چندوجهی ۶

۱۲ خروجی‌های مهم artifacts ۷

۱۳ روش اجرا در VSCode ۷

۱۴ ارتباط با فاز دوم ۸

۱۵ محدودیت‌ها و پیشنهادهای بهبود ۸
۱۶ جمع‌بندی ۸

۱ محل قرارگیری پروژه

کدهای فاز اول در ساختار جداگانه زیر قرار دارند:

```
<PROJECT_ROOT>\phase1
```

داده خام محلی و خروجی‌های تولیدشده فاز اول در همین پوشه نگهداری شده‌اند، اما طبق فایل `.gitignore` وارد git نمی‌شوند:

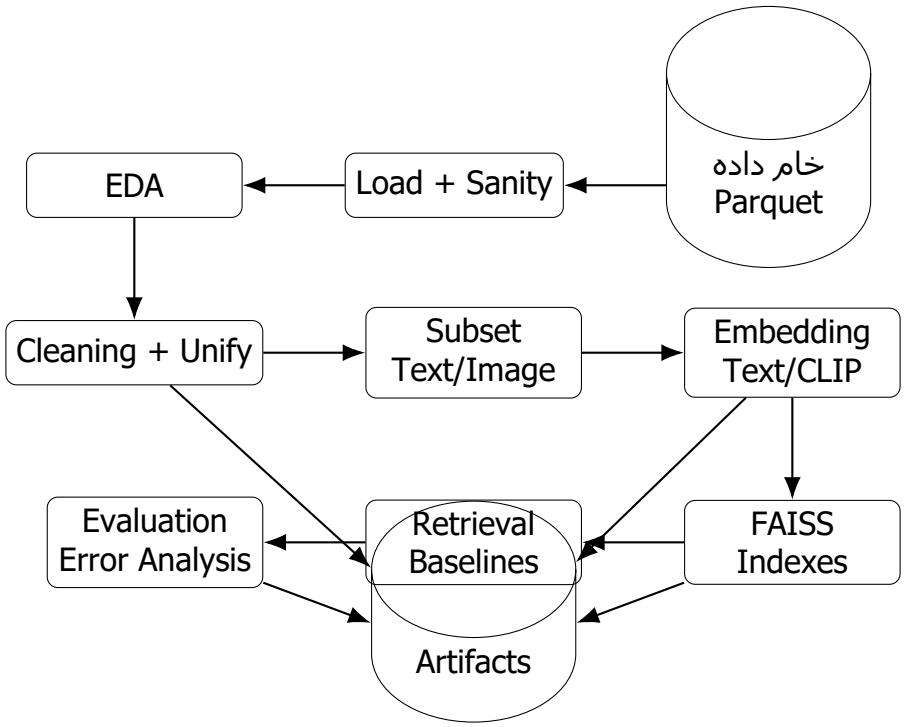
```
phase1/Torob-Database
phase1/torob_rag/artifacts
```

۲ خلاصه نیازمندی‌ها و پیاده‌سازی

بخش مستند فاز اول	پیاده‌سازی انجام‌شده	فایل‌ها و خروجی‌ها
شناخت دیتاست	خواندن همه جدول‌های relational ترب و بررسی تعداد ردیف‌ها، ستون‌ها، کلیدهای یکتا، missing values و روابط کلید خارجی.	checkpoint_01_load_sanity.py، checkpoint_01_sanity.json
EDA	تحلیل کیفیت داده، قیمت‌ها، ناموجودی‌ها، عدم توازن دسته/برند، تعداد فروشنده برای هر Base، طول هاquery و بازه هاtimestamp.	checkpoint_02_eda_report.py، artifacts/eda
Cleaning و preprocess-ing	حذف قیمت‌های نامعتبر، cap کردن outlier قیمت، join با برند/دسته، ساخت متن استاندارد برای embed-ding و تولید جدول unified.	checkpoint_03_clean_unify.py، artifacts/clean
Feature engineering	تولید متن text_for_embedding از نام فارسی، نام انگلیسی، دسته، برند و ویژگی‌های اضافه؛ آماده‌سازی subset متنی و تصویری.	checkpoint_04a*.py، checkpoint_04b_text_embeddings.py
RAG Text-only	embedding متنی با مدل چندزبانه، ساخت FAISS با IndexFlatIP و بازیابی top-k محصولات.	embeddings/text، vector_db/faiss/text
RAG Image-only	انتخاب subset تصویری، دانلود تصاویر، embedding با CLIP، ساخت FAISS تصویری و sanity evaluation.	image/checkpoint_08*.py، embeddings/clip_v1/image
RAG Multimodal	ساخت corpus مشترک متن+تصویر، embedding متن روی محصولات دارای تصویر، و fusion late بین امتیاز متن و تصویر.	multimodal/checkpoint_09*.py
Evaluation	ارزیابی retrieval text با Recall@k، MRR@k و NDCG@k، تحلیل خطا، و evaluation sanity برای image/mul-timodal.	artifacts/eval

بازتولیدپذیری	نگهداری پارامترها در config.yaml، ذخیره همه خروجی‌های check-pointها و ثابت نگه داشتن seed.	configs/config.yaml، requirements.txt

۳ معماری کلی فاز اول



شکل ۱: معماری کلی پایپلاین فاز اول

۴ ساختار داده‌ها

دیتاست فاز اول شامل جدول‌های relational زیر است. دو سطح مهم محصول در پروژه وجود دارد: Base Product که نماینده یک محصول اصلی است، و Member Product که همان محصول را در سطح فروشنده/فروشگاه با قیمت مشخص نشان می‌دهد.

جدول	تعداد ردیف	نقش در پروژه
base_products	۱،۰۲۲،۲۹۸	محصولات پایه شامل random_key، نام فارسی/انگلیسی، برند، دسته، ویژگی‌ها و تصویر.
members	۱،۹۴۸،۶۶۵	محصولات فروشگاهی شامل قیمت، فروشگاه و base_random_key.
searches	۵۸۸،۳۴۷	لاگ جست‌وجوی کاربران و نتایج نمایش‌داده‌شده.
base_views	۱۹۹،۹۱۶	بازدید کاربران از محصولات پایه پس از جست‌وجو.
final_clicks	۱۷،۳۷۱	کلیک نهایی کاربر روی فروشنده/محصول فروشگاهی.

categories	۷۴۶	ساختار دسته‌بندی محصولات.
brands	۲،۰۲۵	اطلاعات برندها.
shops	۲۳،۳۴۲	اطلاعات فروشگاه، شهر، امتیاز و ضمانت.
cities	۶۵۱	نام شهرها.

۵ تحلیل اکتشافی داده

در مرحله EDA، کیفیت داده و الگوهای مهم بررسی شد. مهم‌ترین یافته‌ها:

- نرخ missing در base_products بسیار کم بود؛ برای image_url حدود ۰۰۳۳.۰.
- در جدول members فقط ۲ قیمت صفر یا منفی دیده شد، اما outlier شدید قیمت وجود داشت.
- میانه قیمت فروشنده‌ها ۱،۷۰۰،۰۰۰ و صدک ۹۹ برابر ۹۵،۰۰۰،۰۰۰ بود.
- بیشینه قیمت خام حدود 1.99e17 بود و باید برای جلوگیری از اثر مخرب روی تحلیل و ranking کنترل می‌شد.
- تعداد فروشنده برای هر base در میانه ۱، در صدک ۹۹ برابر ۱۷ و در بیشینه ۱۵۴۸ بود.
- در نمونه ۲۰۰،۰۰۰تایی ،هاquery میانه طول query برابر ۱۳ و صدک ۹۵ برابر ۲۸ کاراکتر بود.
- عدم توازن برند شدید بود؛ سهم ۱۰ برند پرتکرار حدود ۳%.۸۶ گزارش شد.

خروجی‌های EDA در مسیر زیر قرار دارند:

```
phase1/torob_rag/artifacts/eda
```

۶ پاک‌سازی و یکپارچه‌سازی

در این مرحله داده خام به داده قابل استفاده برای retrieval تبدیل شد. تصمیم‌های اصلی:

- قیمت‌های کمتر از ۱ حذف شدند.
- قیمت‌ها با مقدار price_max_cap = 1000000000 کنترل شدند.
- ۴۴۰ ردیف نامعتبر از جدول members حذف شد.
- تعداد هاmember از ۱،۹۴۸،۶۶۵ به ۱،۹۴۸،۲۲۵ رسید.
- برای همه ۱،۰۲۲،۲۹۸ محصول پایه متن embedding ساخته شد و bp_text_empty = 0 بود.

خروجی‌های اصلی:

```
phase1/torob_rag/artifacts/clean/base_products_clean.parquet
phase1/torob_rag/artifacts/clean/members_clean.parquet
phase1/torob_rag/artifacts/clean/unified_base_products.parquet
phase1/torob_rag/artifacts/clean/cleaning_summary.json
```

۷ استراتژی subset

به دلیل محدودیت CPU و حجم بیش از یک میلیون محصول، برای embedding متنی یک subset قابل دفاع ساخته شد. نسخه نهایی subset با وزن‌دهی به لاگ‌ها ساخته شد:

- نسخه: v2_logs_weighted
- اندازه subset: ۲۰۰،۰۰۰ محصول پایه
- وزن impression: ۱
- وزن view: ۵
- وزن final: click ۲۰

این روش نسبت به انتخاب تصادفی بهتر است، چون محصولاتی را نگه می‌دارد که در رفتار واقعی کاربران بیشتر دیده یا کلیک شده‌اند. با این حال، در گزارش خطا مشاهده شد که پوشش برخی هاintent مثل بعضی هایquery موبایل هنوز پایین است؛ بنابراین بخشی از خطاها به محدودیت پوشش subset مربوط است.

۸ بازیابی متنی

برای baseline متنی از مدل چندزبانه زیر استفاده شد:

```
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
```

مشخصات embedding متنی:

- تعداد محصولات: ۲۰۰،۰۰۰
- بعد embedding: ۳۸۴
- batch_size: ۶۴
- max_length: ۱۲۸
- normalization فعال بود.

برای جست‌وجو از FAISS با product inner روی بردارهای normalize شده استفاده شد؛ در این حالت product inner معادل similarity cosine عمل می‌کند.
فایل‌های اصلی:

```
phase1/torob_rag/src/torob_rag/embeddings/checkpoint_04b_text_embeddings.py
phase1/torob_rag/src/torob_rag/vector_db/checkpoint_05a_build_faiss_text.py
phase1/torob_rag/src/torob_rag/eval/checkpoint_06_text_only_eval.py
```

۹ ارزیابی بازیابی متنی

ارزیابی با truth ground از base_views انجام شد؛ یعنی محصولاتی که کاربر پس از جست‌وجو واقعا دیده است. فقط نمونه‌هایی استفاده شدند که حداقل یک item clicked داخل subset داشتند.

متریک	مقدار	توضیح
eval_samples	۵،۰۰۰	تعداد جست‌وجوهای ارزیابی‌شده.
recall@1	۰۰۶۲.۰	درصد مواردی که جواب درست در رتبه اول بود.

recall@5	۰۲۱۶.۰	درصد مواردی که جواب درست در پنج نتیجه اول بود.
recall@10	۰۳۲۸.۰	درصد مواردی که جواب درست در ده نتیجه اول بود.
mrr@10	۰۱۲۵۴.۰	میانگین rank reciprocal تا رتبه ۱۰.
ndcg@10	۰۱۴۰۶.۰	کیفیت رتبه‌بندی با در نظر گرفتن جایگاه نتیجه درست.
failures	۴،۸۳۶	تعداد نمونه‌هایی که نتیجه درست در top-k دیده نشد.

تحلیل خطا نشان داد هایquery کوتاه و عمومی ambiguity بالایی دارند و بعضی هایquery ترکیبی به سمت کلمات غالب drift می‌کنند. نمونه‌های قابل خواندن خطا در فایل زیر ذخیره شده‌اند:

```
phase1/torob_rag/artifacts/eval/text/failure_examples_readable.csv
```

۱۰ بازیابی تصویری

برای baseline تصویری ابتدا ۲۰،۰۰۰ محصول دارای تصویر و view بالا انتخاب شد. سپس تصاویر دانلود و با embedding CLIP شدند.

- محصولات انتخاب‌شده برای subset تصویری: ۲۰،۰۰۰
- تعداد دانلود موفق: ۱۶،۲۳۶
- تعداد دانلود ناموفق: ۳،۷۶۴
- مدل تصویر: `openai/clip-vit-base-patch32`
- بعد embedding: ۵۱۲
- `batch_size`: ۳۲
- normalization فعال بود.

evaluation Sanity تصویری نشان داد:

- `self_recall@1 = 0.993964`
- میانه `top1_score` برابر ۰.۱ بود.

این تست sanity است، چون query همان embedding تصویر محصول است؛ بنابراین انتظار می‌رود خود محصول در رتبه اول برگردد.

۱۱ بازیابی چندوجهی

برای بخش امتیازی چندوجهی، corpus مشترک روی محصولاتی ساخته شد که تصویر آن‌ها با موفقیت دانلود و embedding شده بود.

- تعداد آیتم‌های corpus مشترک: ۱۶،۲۳۶
- embedding متن برای همین corpus با مدل `paraphrase-multilingual-MiniLM-L12-v2`
- بعد embedding متن: ۳۸۴
- روش ترکیب: fusion late

فرمول ترکیب امتیاز:

```
score = alpha_text * text_score + (1 - alpha_text) * image_score
```

در evaluation sanity مقدار alpha_text = 0.5، pool_text = 200 و pool_image = 200 استفاده شد. برای نمونه ۲،۰۰۰تایی، recall@1، recall@5، recall@10 و mrr@10 همگی برابر ۰.۱ بودند. این نیز test sanity است، نه ارزیابی واقعی روی لاگ multimodal کاربر.

۱۲ خروجی‌های مهم artifacts

مسیر	محتوا
artifacts/checkpoint_01_sanity.json	خروجی check sanity جدول‌ها و روابط.
artifacts/eda	نمودارها و خلاصه EDA.
artifacts/clean	داده پاک‌سازی‌شده و unified برای فازهای بعد.
artifacts/subset	های subset متنی و خلاصه انتخاب آن‌ها.
artifacts/embeddings/text	embedding و metadata index متنی.
artifacts/vector_db/faiss/text	index FAISS متنی.
artifacts/embeddings/clip_v1/image	embedding و metadata index تصویری.
artifacts/vector_db/faiss/clip_v1/image	index FAISS تصویری.
artifacts/multimodal/clip_v1	corpus مشترک متن/تصویر.
artifacts/eval	گزارش‌های ارزیابی و تحلیل خطا.

۱۳ روش اجرا در VSCode

پوشه فاز اول را در VSCode باز کنید:

```
<PROJECT_ROOT>\phase1
```

سپس در ترمینال PowerShell:

```
cd "<PROJECT_ROOT>\phase1"
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
$env:PYTHONPATH = ".\torob_rag\src"
```

نمونه اجرای checkpointها:

```
python .\torob_rag\src\torob_rag\eda\checkpoint_01_load_sanity.py
python .\torob_rag\src\torob_rag\eda\checkpoint_02_eda_report.py
python .\torob_rag\src\torob_rag\cleaning\checkpoint_03_clean_unify.py
python .\torob_rag\src\torob_rag\eda\
   checkpoint_04a2_build_subset_from_searches.py
python .\torob_rag\src\torob_rag\embeddings\checkpoint_04b_text_embeddings.
   py
python .\torob_rag\src\torob_rag\vector_db\checkpoint_05a_build_faiss_text.
   py
python .\torob_rag\src\torob_rag\eval\checkpoint_06_text_only_eval.py
```

برای بخش تصویر و چندوجهی:

```
python .\torob_rag\src\torob_rag\image\checkpoint_08a_build_image_subset.py
python .\torob_rag\src\torob_rag\image\checkpoint_08b_download_images.py
python .\torob_rag\src\torob_rag\image\checkpoint_08c_clip_image_embeddings
   .py
python .\torob_rag\src\torob_rag\image\checkpoint_08d_build_faiss_image.py
python .\torob_rag\src\torob_rag\image\checkpoint_08f_image_index_sanity.py
python .\torob_rag\src\torob_rag\multimodal\
   checkpoint_09a_prepare_mm_corpus.py
python .\torob_rag\src\torob_rag\multimodal\checkpoint_09b_embed_mm_text.py
python .\torob_rag\src\torob_rag\multimodal\
   checkpoint_09c_build_faiss_mm_text.py
python .\torob_rag\src\torob_rag\multimodal\
   checkpoint_09e_multimodal_sanity_eval.py
```

۱۴ ارتباط با فاز دوم

فاز دوم به همین خروجی‌های فاز اول وابسته است. مهم‌ترین فایل‌هایی که عامل فاز دوم استفاده می‌کند:

```
phase1/torob_rag/artifacts/clean/unified_base_products.parquet
phase1/torob_rag/artifacts/clean/members_clean.parquet
phase1/torob_rag/artifacts/embeddings/clip_v1/image/image_index.parquet
```

بنابراین فاز اول نقش لایه داده و retrieval را دارد و فاز دوم روی آن ،API حافظه کوتاه‌مدت و منطق agentic اضافه می‌کند.

۱۵ محدودیت‌ها و پیشنهادهای بهبود

- عملکرد retrieval text-only روی لاگ واقعی پایین است؛ علت‌های اصلی شامل ambi-guity های‌query کوتاه، پوشش محدود subset و نبود reranker قوی‌تر است.
- برای بهبود متن می‌توان از های‌retrieval-tuned encoder مانند ،E۵ boost reranking، واژگانی و افزایش پوشش subset استفاده کرد.
- ارزیابی تصویر و multimodal فعلی بیشتر check sanity است و بهتر است در نسخه بعدی روی داده واقعی query/image یا سناریوهای دستی دقیق‌تر ارزیابی شود.
- metadata بعضی های‌artifact قدیمی ممکن است مسیرهای زمان تولید را نگه داشته باشند؛ مسیر اجرایی فعلی پروژه روی درایو D است.
- برای production بهتر است versioning رسمی برای embedding ،هاrun lockfile وابستگی‌ها، و تست‌های کوچک regression اضافه شود.

۱۶ جمع‌بندی

فاز اول یک پایپلاین کامل و قابل اجرا برای آماده‌سازی داده و ساخت های‌RAG baseline محصول فراهم می‌کند. خروجی‌های پاک‌سازی، ،هاFAISS embedding هاindex و گزارش‌های ارزیابی

در artifacts ذخیره شده‌اند و همین خروجی‌ها مبنای فاز دوم هستند. با وجود محدودیت‌های CPU و ،subset ساختار محورcheckpoint پروژه باعث می‌شود مراحل کار قابل بازتولید، قابل توضیح و قابل توسعه باشند.